

Contents

Case Study 1: Process and Thread Management in Real-Time Systems1

1.1 Introduction	1
1.2 What is RTOS?	1
1.3 What is a Process?	1
1.4 What is a Thread?	1
1.5 Difference between Process and Thread	2
1.6 Process Management	2
1.7 Thread Management	3
1.8 Scheduling in RTOS	3
1.8.1 Rate Monotonic Scheduling (RMS)	3
1.8.2 Earliest Deadline First (EDF)	3
1.9 Synchronization	3
1.9.1 Mutex	3
1.9.2 Semaphore	3
1.10 Deadlocks in RTOS	4
1.11 Real-Life Case Study: Hospital Monitoring System.....	4
1.12 Advantages	5
1.13 Challenges	5
1.14 Summary	5

Case Study 2: Operating Systems for the Internet of Things (IoT)6

2.1 Introduction	6
2.2 What is IoT?	6
2.3 Why IoT Needs an Operating System.....	6
2.4 Features of IoT Operating Systems	6
2.5 IoT Architecture	6
2.6 Process Management	7
2.7 Memory Management.....	7
2.8 Communication.....	7
2.9 Security	7
2.10 Popular IoT Operating Systems	7
2.10.1 FreeRTOS	7
2.10.2 Zephyr.....	7
2.10.3 RIOT OS	8
2.10.4 Contiki-NG	8
2.10.5 TinyOS.....	8
2.11 Case Study: Smart Home System.....	9

2.12 Advantages	10
2.13 Challenges	10
2.14 Summary	10

Case Study 3: Operating System Support for Mobile and Embedded Devices.....**11**

3.1 Introduction	11
3.2 Mobile Operating Systems	11
3.3 Embedded Operating Systems	11
3.4 Android	11
3.5 iOS.....	11
3.6 Embedded Linux	12
3.7 FreeRTOS in Embedded Devices.....	12
3.8 Memory Management.....	12
3.9 Process Management	12
3.10 Thread Management	12
3.11 Battery Optimization.....	12
3.12 Security	13
3.13 Real-Life Case Study: Smartphone Operating System.....	13
3.14 Comparison Tables	14
3.15 Advantages	15
3.16 Challenges	15
3.17 Summary	15

References.....16

Case Study 1: Process and Thread Management in Real-Time Systems

1.1 Introduction

An operating system manages the work a computer has to do. It decides which program runs, when it runs, and how CPU time and memory are shared. For a normal laptop, a small delay is usually harmless. In a real-time system, timing can be critical. Devices such as ventilators, car airbag controllers, and drone controllers may need to react within a fixed time.

This case study explains how an RTOS manages processes and threads, chooses the next task, and keeps shared data safe. A hospital monitoring system is used as the main example.

1.2 What is RTOS?

A Real-Time Operating System (RTOS) is designed to complete important tasks within a known deadline. The key idea is predictable timing, not simply maximum speed. RTOS systems are commonly described as hard or soft real-time. A hard real-time system must meet its deadline because missing it can cause damage or injury. A soft real-time system can tolerate an occasional late result, such as a dropped frame in a video call.

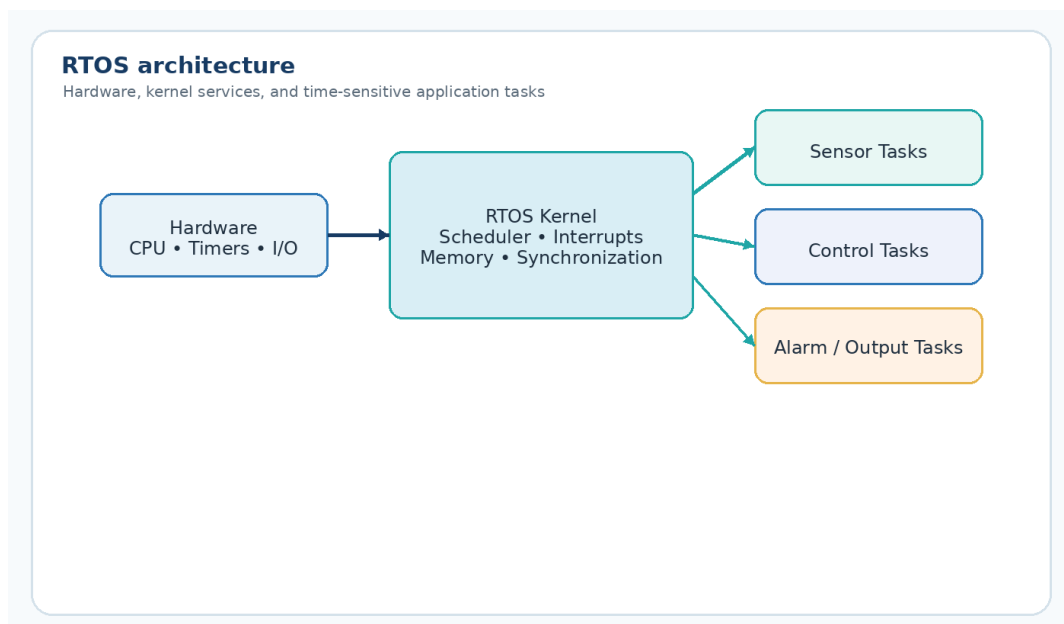


Figure 1: RTOS Architecture showing the kernel between hardware and application tasks

1.3 What is a Process?

A process is a running program together with the resources it needs, such as memory, open files, and register values. For example, when a washing machine starts a cycle, the control software runs as a process and keeps the current cycle information.

Separate processes normally have separate memory spaces. Because of this separation, a problem in one process usually does not directly affect another process.

1.4 What is a Thread?

A thread is a smaller unit of work inside a process. Several threads can belong to one process and share that process memory, which makes thread switching relatively lightweight.

A smart home hub is a simple example. One process can contain a thread for the door sensor, another for temperature, and another for sending updates to a phone. These threads can use the same shared data.

1.5 Difference between Process and Thread

The table compares the main practical differences between processes and threads.

Point	Process	Thread
Memory	Has its own separate memory space	Shares memory with other threads in the same process
Startup time	Usually slower to create	Usually faster to create
Communication	Needs inter-process communication (IPC)	Threads can share process data directly
Dependency	Independence of other processes	Runs within its parent process
Crash effect	A crash usually does not affect other processes	A crash can affect the process and its threads
Example	Two separate apps, such as Calculator and Camera	Two parts of one app, such as typing and spell-check

Table 1: Difference between Process and Thread

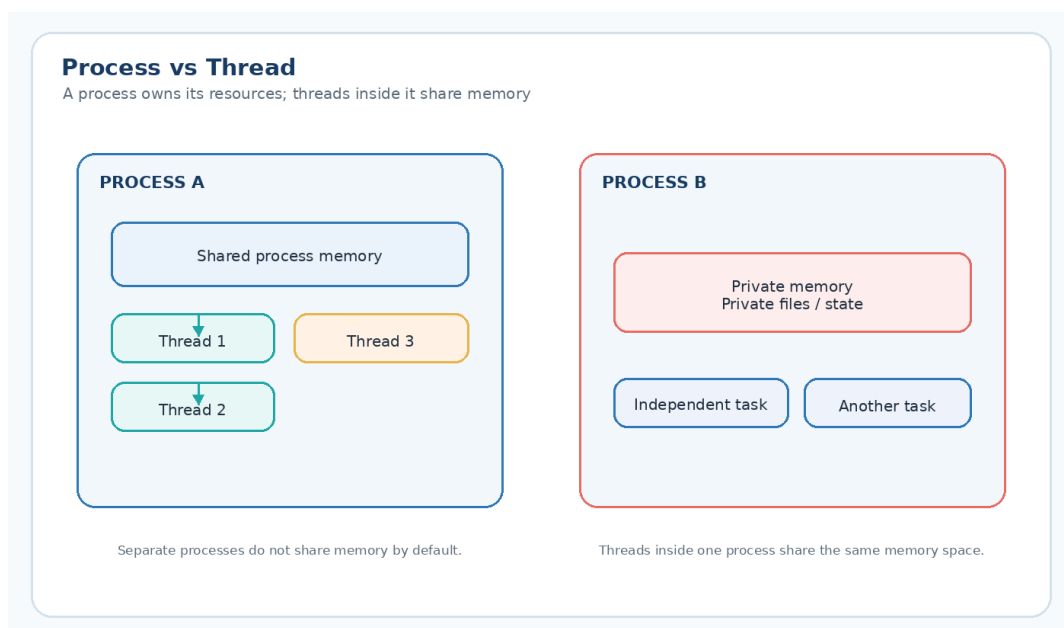


Figure 2: Process vs Thread: threads inside one process share memory, separate processes do not

1.6 Process Management

The OS keeps a Process Control Block (PCB) for each process. It records details such as the process ID, current state, program counter, and memory information. This lets the OS pause a process and later continue it.

A process moves through several common states during its life.

A process usually moves through these states during its life:

- New — the process is being created.
- Ready — the process is waiting for CPU time.
- Running — the process is currently executing.
- Waiting — the process is waiting for an event, input, or sensor result.
- Terminated — the process has finished.

1.7 Thread Management

Thread management is similar, but switching between threads is usually cheaper because threads in one process already share memory. This is useful in small real-time devices where many short tasks need to run.

Some RTOS designs use a thread pool. A set of threads is created in advance and reused for new work, which avoids the cost of creating a new thread every time a task appears.

1.8 Scheduling in RTOS

Scheduling decides which task or thread should run next. In an RTOS, the decision must be quick and predictable because a poor choice can cause a missed deadline.

1.8.1 Rate Monotonic Scheduling (RMS)

RMS assigns fixed priorities using task periods. A task that runs more frequently, and therefore has a shorter period, receives a higher priority. In a hospital monitor, a fast heartbeat check can receive higher priority than a temperature check that runs every few seconds.

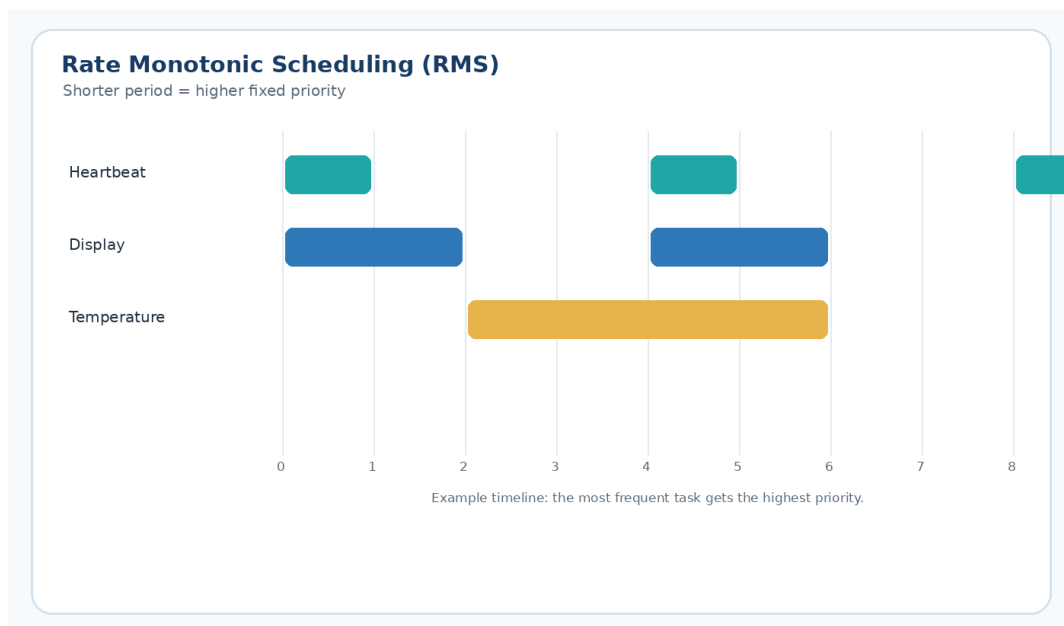


Figure 3: Example timeline of Rate Monotonic Scheduling with three tasks of different periods

1.8.2 Earliest Deadline First (EDF)

EDF changes priorities as time moves forward. The task with the nearest deadline gets the CPU first. It can use the processor efficiently, although its behaviour is more dynamic and can be harder to analyse.

1.9 Synchronization

When several threads use the same data, two threads may update it at the same time. Synchronization tools help prevent these conflicts.

1.9.1 Mutex

A mutex acts like one shared key. Only the thread holding the key can enter the protected section, and it releases the key when it is finished. This prevents two tasks from using the same resource at the same time.

1.9.2 Semaphore

A semaphore represents a limited number of available slots. If a resource has three slots, up to three threads can use it at once. When all slots are busy, another thread waits until one becomes available.

1.10 Deadlocks in RTOS

A deadlock occurs when tasks wait for one another and none can continue. The usual conditions are mutual exclusion, hold and wait, no preemption, and circular wait.

In a real-time device, a deadlock can also become a timing problem. For example, if the heart-rate task waits for a display lock while the alarm task waits for heart-rate data, both can stop and the monitor may miss an important update.

1.11 Real-Life Case Study: Hospital Monitoring System

A bedside monitor is a useful real-life RTOS example. It reads heart rate, oxygen level (SpO2), and blood pressure, displays the latest values, and raises an alarm when a reading passes a critical limit.

Each type of sensor data can be handled by a separate thread. A fast ECG task may run every 10 milliseconds, while blood pressure may update much less often. A shared data area stores the latest readings, with a mutex used to protect updates.

The alarm task receives very high priority because detecting and reporting a dangerous reading is more urgent than refreshing a normal graph. This shows how scheduling rules are connected to real device behaviour.

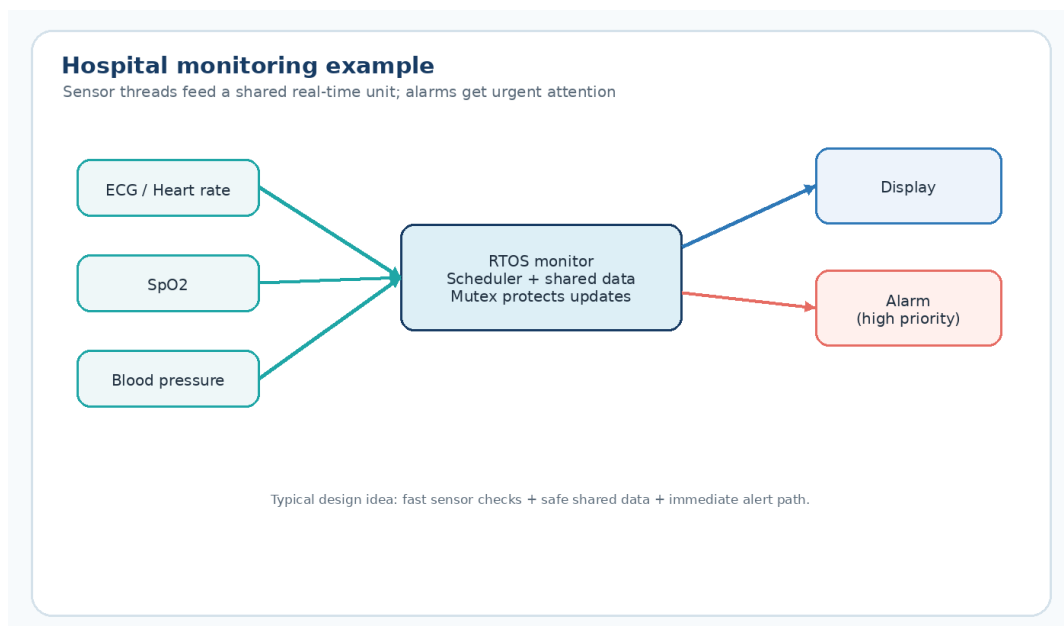


Figure 4: Hospital monitoring system: sensor threads feed a shared RTOS unit that drives the display and alarm

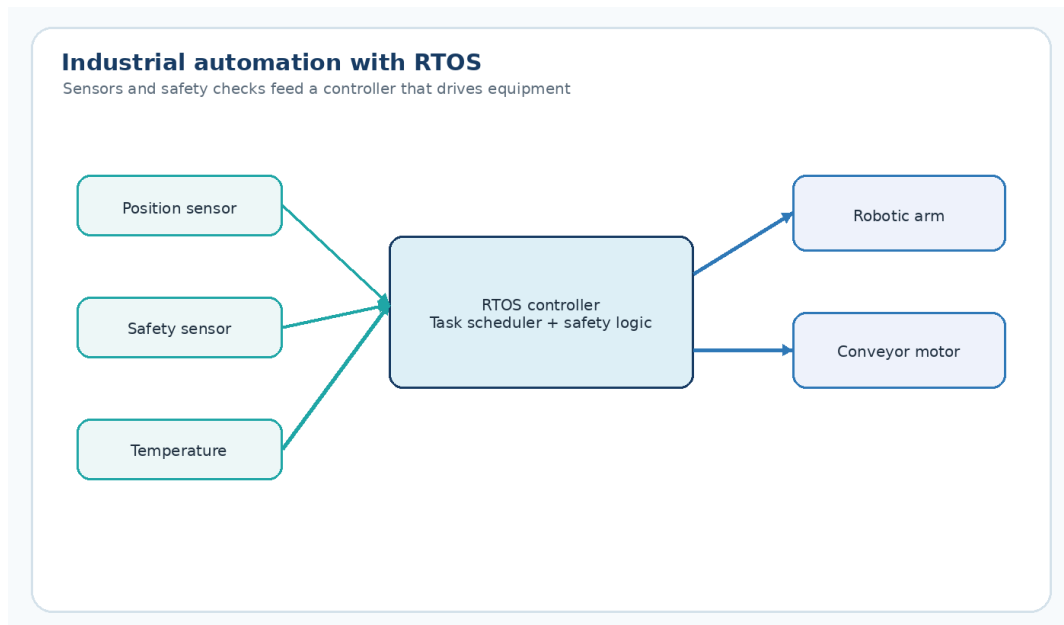


Figure 5: Industrial automation is another common use of RTOS, where sensors and safety checks control a robotic arm

The same idea appears in other systems, such as traffic lights that change on a strict schedule or vehicle controllers that must process sensor data quickly enough to react safely.

1.12 Advantages

- Predictable response time is important for safety-critical tasks.
- The system can work with small processors and limited memory.
- RTOS designs can handle many sensor inputs and interrupts efficiently.
- A slow task is less likely to block unrelated work when priorities are designed well.

1.13 Challenges

- Task priorities must be planned carefully.
- Timing bugs can be harder to reproduce than ordinary software bugs.
- Adding a new task can change the timing of existing tasks.
- Poor synchronization can create difficult-to-reproduce deadlocks.

1.14 Summary

Overall, an RTOS gives the system predictable timing. Processes keep separate memory while threads share memory inside a process. RMS and EDF help choose tasks, while mutexes and semaphores protect shared data. The hospital monitor shows how these ideas work together in practice.

Case Study 2: Operating Systems for the Internet of Things (IoT)

2.1 Introduction

IoT connects everyday objects that have sensors, processors, and network links. Smart bulbs, fitness bands, and door locks are small computers, and many of them need an operating system that is lightweight and power aware.

2.2 What is IoT?

The Internet of Things (IoT) is a group of physical devices that collect data and communicate with other devices or cloud services. A soil-moisture sensor that can start an irrigation pump automatically is a simple example.

2.3 Why IoT Needs an Operating System

A tiny IoT device can run one program without an OS, but that becomes harder when the device must read sensors, manage timers, communicate over a network, and save power at the same time. An IoT OS provides a cleaner way to manage these jobs.

2.4 Features of IoT Operating Systems

- Small memory footprint, because many IoT chips have very limited RAM.
- Low power use, so battery-powered devices can run for long periods.
- Real-time behaviour for time-sensitive sensor or control tasks.
- Wireless support for standards such as Wi-Fi, Bluetooth Low Energy, and Zigbee.
- Reliable operation for devices that may run for long periods without human attention.

2.5 IoT Architecture

IoT architecture is often shown in four layers. The perception layer contains sensors and actuators. The network layer moves data. The processing layer runs the OS and local logic. The application layer is the dashboard or mobile interface used by people. where the operating system and any local logic run. At the top is the application layer, which is the dashboard or mobile app a person actually sees and uses.

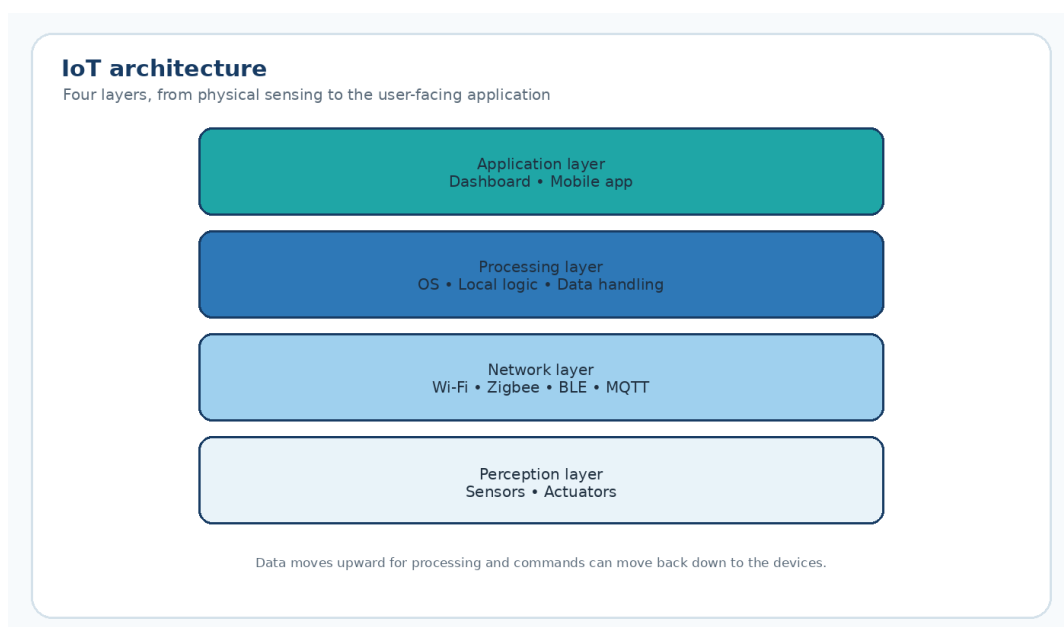


Figure 6: Layered architecture of a typical IoT system

2.6 Process Management

Because IoT chips have limited memory, many IoT systems prefer lightweight tasks or threads instead of multiple full processes with separate memory spaces. Sharing one memory space reduces overhead.

2.7 Memory Management

Static memory allocation is common in small IoT devices. The amount of memory is planned before the program runs, which reduces the chance of unexpected memory shortage during operation.

2.8 Communication

IoT communication usually uses lightweight protocols. MQTT is common for small messages, while CoAP is designed for constrained devices. Zigbee and Bluetooth Low Energy are also useful for short-range links.

2.9 Security

IoT security matters because a compromised camera, lock, or sensor can affect privacy or safety. Secure boot and encrypted communication are common protections. Low-cost devices may still leave out some of these protections to reduce costs, which is one of the biggest weak points of the whole IoT industry today.

2.10 Popular IoT Operating Systems

2.10.1 FreeRTOS

FreeRTOS is a small open-source RTOS used in many embedded devices. It provides a scheduler, queues, and synchronization tools while keeping memory use low.

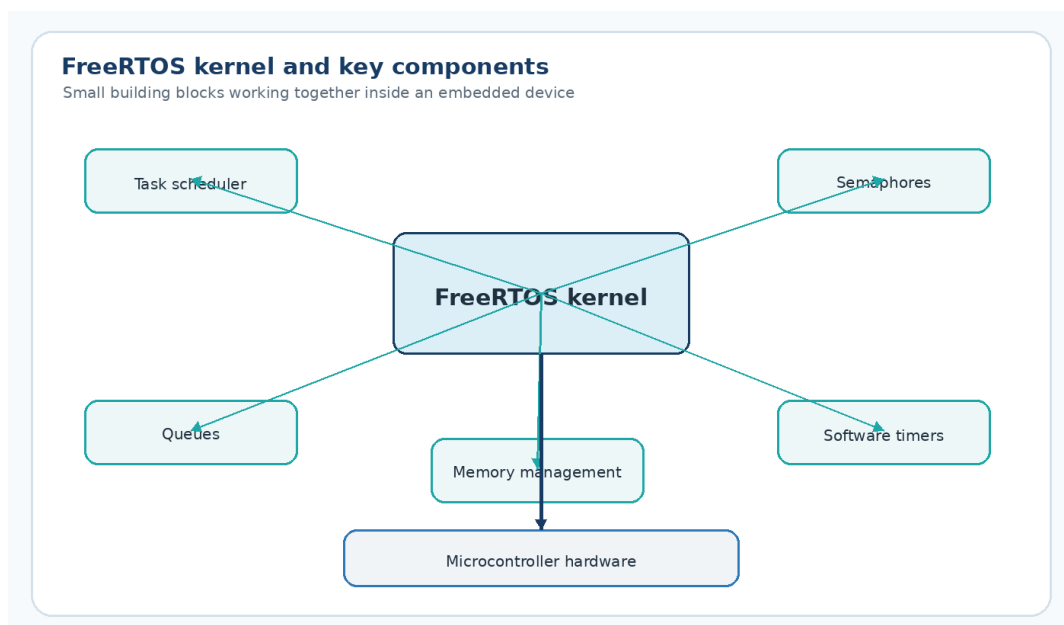


Figure 7: Block diagram of the FreeRTOS kernel and its main components

2.10.2 Zephyr

Zephyr is an open-source RTOS supported by the Linux Foundation. It runs on many types of hardware and includes networking features, making it useful for IoT products that need Bluetooth or Wi-Fi.

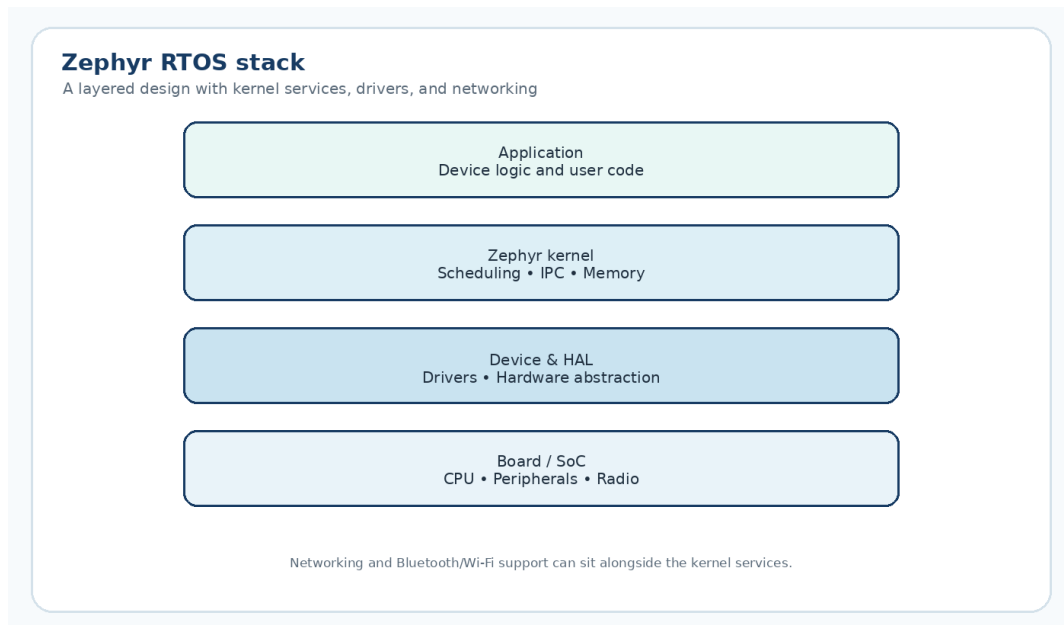


Figure 8: Layered architecture of the Zephyr real-time operating system

2.10.3 RIOT OS

RIOT OS targets very constrained devices and supports standard internet protocols such as IPv6, so small sensor nodes can communicate directly over IP networks.

2.10.4 Contiki-NG

Contiki-NG focuses on low-power wireless and is often used in sensor-network research. Its support for mesh networking lets several small nodes pass data toward a gateway.

2.10.5 TinyOS

TinyOS is an older operating system for wireless sensor networks. Its event-driven approach runs code when an event occurs, which can save power on tiny battery-operated devices.

OS	Memory Footprint	License	Typical Use
FreeRTOS	Very small (a few KB)	Open source (MIT)	Smart plugs, wearables, small sensors
Zephyr	Small to medium	Open source (Apache 2.0)	Devices needing built-in Wi-Fi/Bluetooth
RIOT OS	Very small	Open source (LGPL)	Constrained sensors with direct internet access
Contiki-NG	Small	Open source (BSD)	Low-power mesh sensor networks
TinyOS	Very small	Open source (BSD)	Wireless sensor network research

Table 2: Comparison of popular IoT operating systems

2.11 Case Study: Smart Home System

A smart home is a clear example of several IoT devices working together. A typical setup includes a central hub plus smart lights, a thermostat, a door lock, and a security camera connected through Wi-Fi or Zigbee.

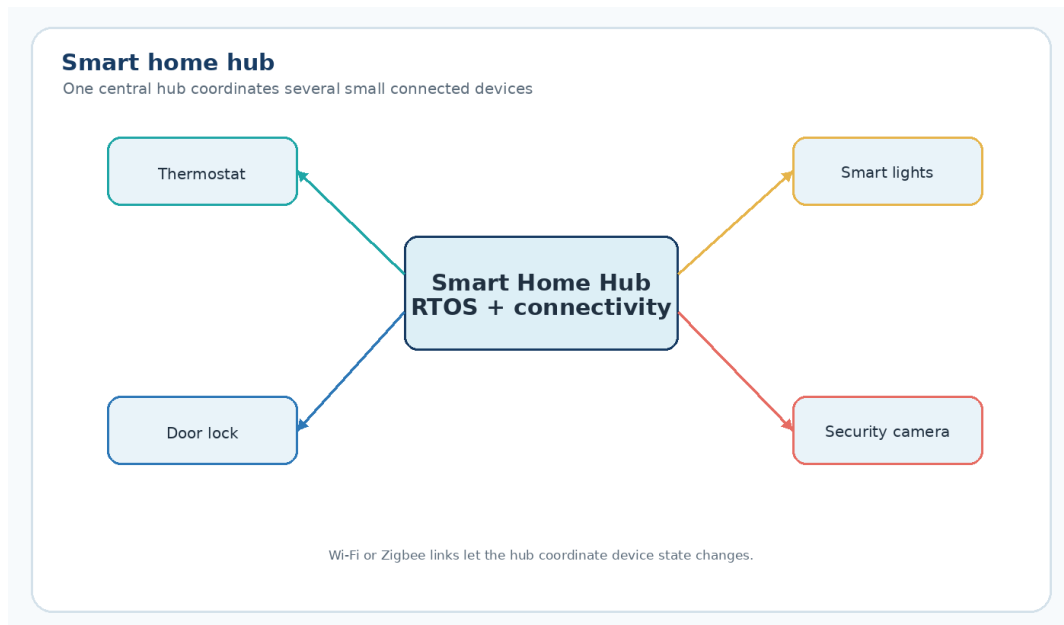


Figure 9: A smart home hub connected to thermostat, lights, door lock and camera

Inside the hub, an RTOS such as FreeRTOS or Zephyr can run several small threads. One may handle phone commands, another may watch the door lock, and another may manage Wi-Fi. Because the threads share the hub memory, a change such as switching from Home to Away can be applied quickly.

The individual devices are usually simpler. A smart bulb or door lock may only need to read a state, perform one action, and report the result to the hub.

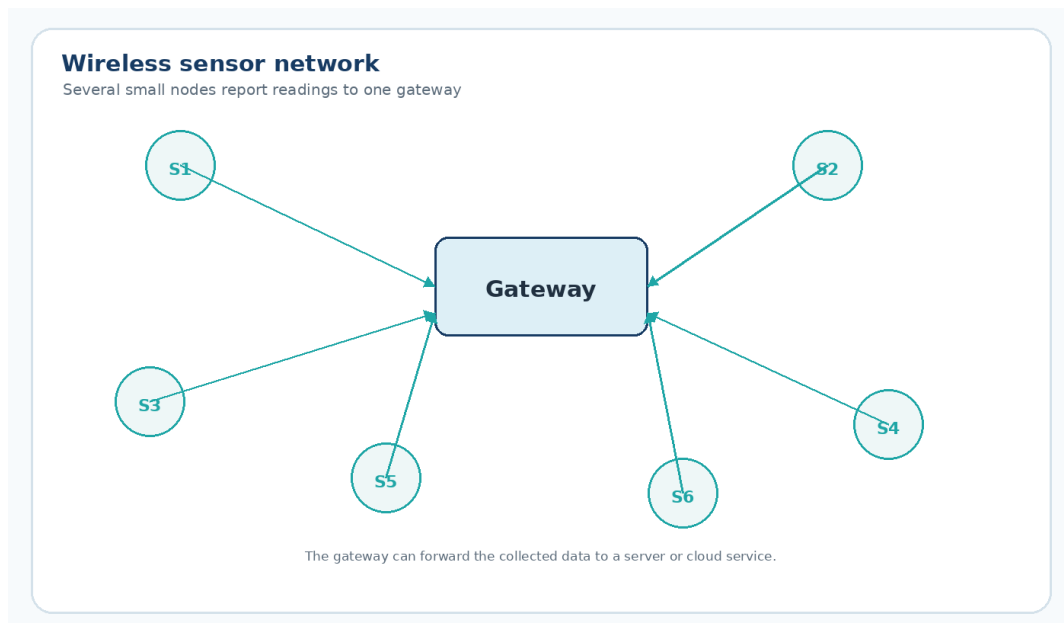


Figure 10: A wireless sensor network where several sensor nodes send data to a central gateway

The same gateway pattern is useful outside the home. In smart agriculture, soil and weather sensors can report to one gateway, which can then trigger an irrigation pump and reduce manual work.

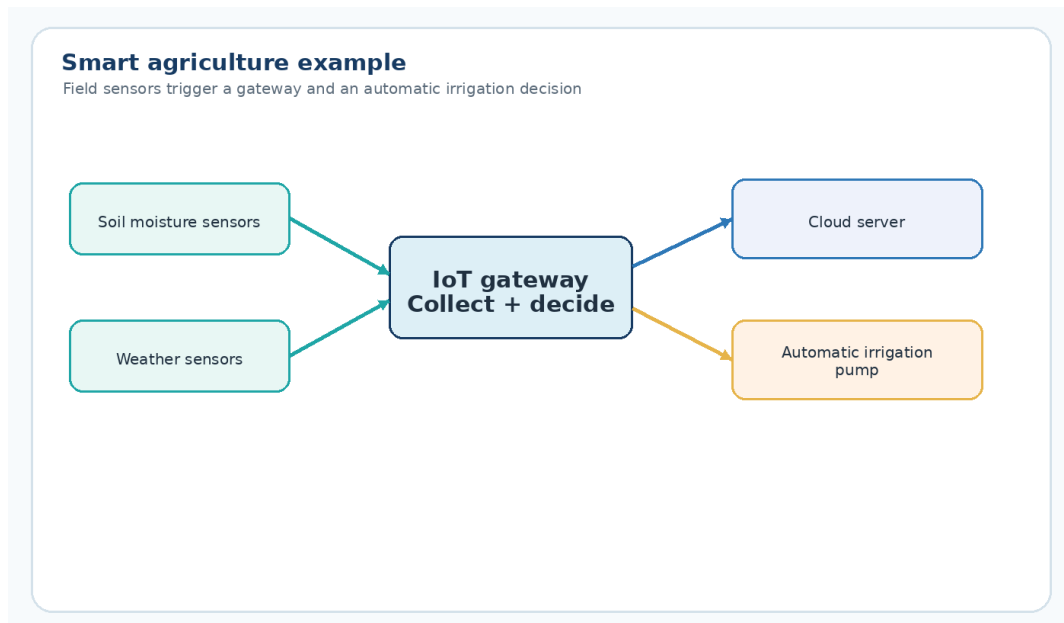


Figure 11: A smart agriculture setup using field sensors, a gateway, and an automatic irrigation pump

2.12 Advantages

- Very low power use helps battery-powered devices last longer.
- Small size allows the OS to fit on tiny chips.
- Wireless protocols are widely supported.
- Many IoT operating systems are open source, which can reduce development cost.

2.13 Challenges

- Limited memory can make later feature additions difficult.
- Security can be weaker than on larger systems if device resources are very limited.
- Different IoT operating systems may not be compatible with one another.
- Large deployments need careful planning for connectivity, updates, and device management.

2.14 Summary

IoT operating systems focus on being small, power-efficient, and responsive. FreeRTOS, Zephyr, RIOT OS, Contiki-NG, and TinyOS use different designs but target similar resource-constrained devices. The smart home and smart agriculture examples show how many small devices can work together as one connected system.

Case Study 3: Operating System Support for Mobile and Embedded Devices

3.1 Introduction

Mobile phones and embedded devices both depend on operating systems, but their needs are different. A smartphone must support many apps, touch input, connectivity, security, and long battery life. An embedded device often has fewer jobs but tighter memory and power limits.

3.2 Mobile Operating Systems

A mobile OS runs on phones and tablets. It manages touch input, cameras, GPS, mobile data, apps, security, and device resources. Android and iOS are the main mobile platforms discussed here.

3.3 Embedded Operating Systems

An embedded OS is built into a device for one main job or a small set of jobs. Examples include microwave controllers, car dashboards, smart TVs, and other dedicated systems.

3.4 Android

Android uses a layered design built around the Linux kernel. The kernel manages hardware, memory, and security. Above are the hardware abstraction layer, runtime and libraries, the Java API framework, and the applications used by the user.

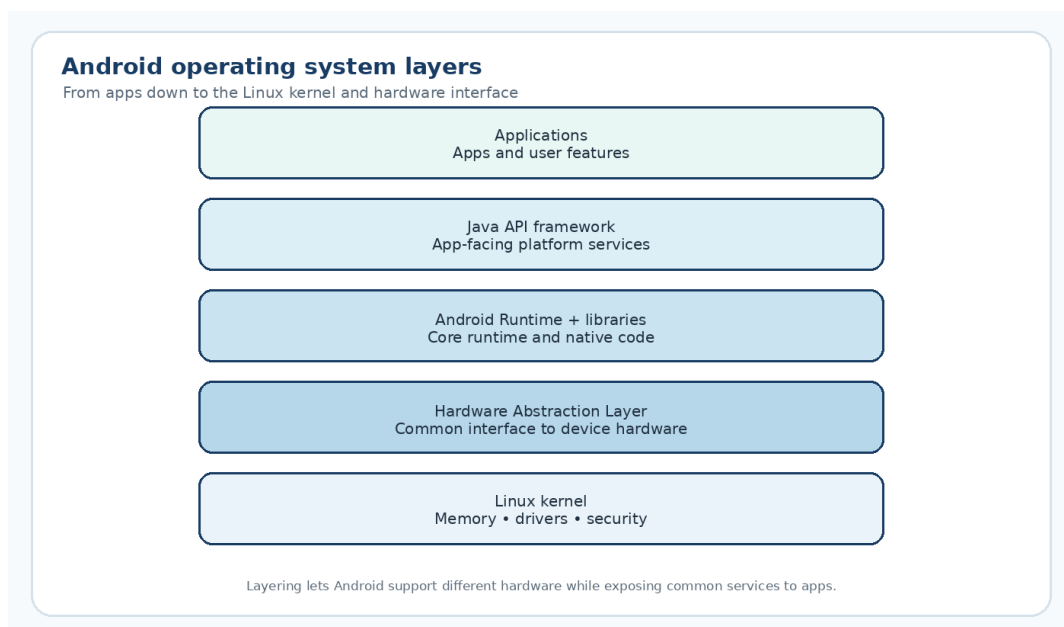


Figure 12: Layered architecture of the Android operating system

3.5 iOS

iOS also uses layers. Core OS handles low-level system work, Core Services provides common services such as networking and location, Media handles graphics and audio, and Cocoa Touch supports the user interface and touch input.

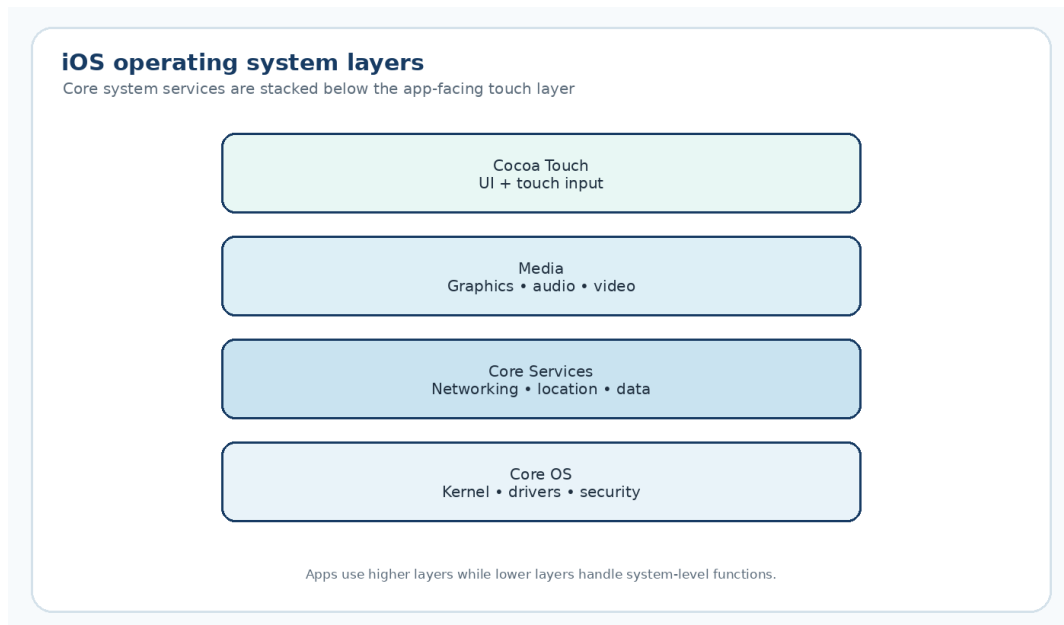


Figure 13: Layered architecture of the iOS operating system

3.6 Embedded Linux

Embedded Linux is a smaller, device-focused Linux system. It is often used when a device has more resources than a microcontroller but still needs a tailored operating system, such as a smart TV, home router, or car infotainment system.

3.7 FreeRTOS in Embedded Devices

FreeRTOS is also useful in embedded products that need real-time behaviour but do not need the size of a full Linux system. Examples include low-power wearable controllers and small automotive control units.

3.8 Memory Management

Mobile operating systems use virtual memory so applications can work with their own logical memory spaces. Android also performs automatic memory cleanup for data that is no longer needed. Small embedded systems often use planned or static memory instead because they may not have enough resources for heavier memory-management features.

3.9 Process Management

On a phone, each app commonly runs in its own process, which helps isolate failures. When memory becomes low, the OS may reduce or stop background work. Embedded systems often use fewer processes and simpler task structures.

3.10 Thread Management

Mobile apps use multiple threads to keep the interface responsive. For example, one thread can handle screen updates while another downloads messages. An embedded device can use a similar pattern for display updates and sensor readings.

3.11 Battery Optimization

Battery life is protected by limiting background activity, reducing screen brightness or refresh work when idle, and lowering processor activity when full performance is not needed.

3.12 Security

Mobile systems use permissions, sandboxing, and encrypted storage to protect data. Embedded systems may have fewer security features, but important devices can still use secure boot and signed updates.

3.13 Real-Life Case Study: Smartphone Operating System

Consider a phone running a video call while a low-battery warning appears. The video call, battery monitor, and notification system may run as separate processes while sharing Android services underneath.

The video call can use separate threads for the camera, microphone, network transfer, and touch controls. The scheduler gives the foreground call the resources it needs while background tasks can be slowed down.

When the battery level reaches a threshold, the power manager can trigger a notification without stopping the active call. This is a practical example of process isolation, threads, and scheduling working together.

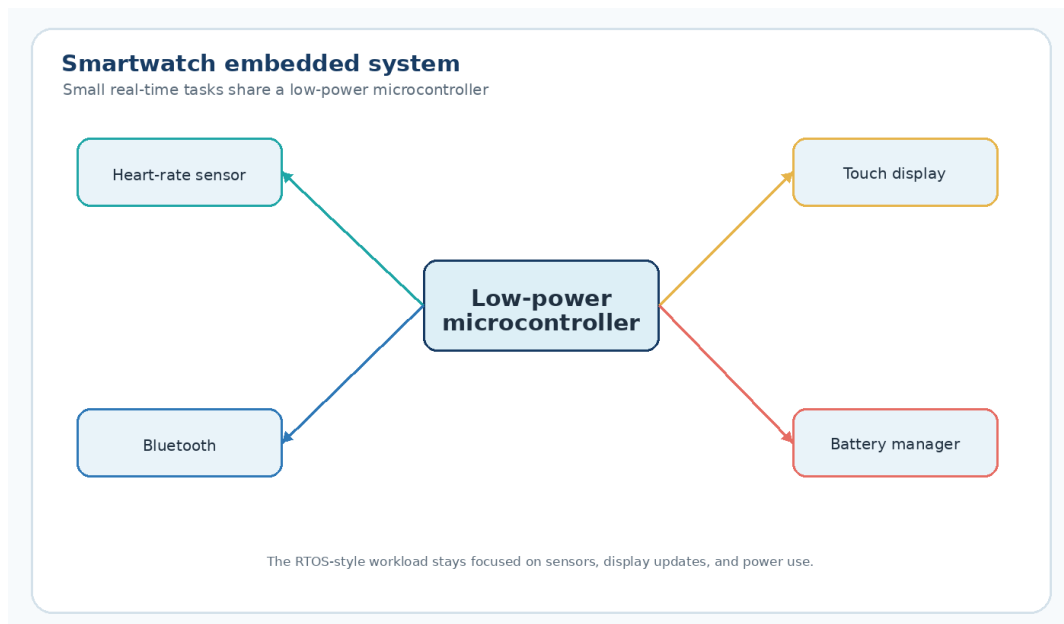


Figure 14: Internal components of a smartwatch, another embedded device built on similar OS principles

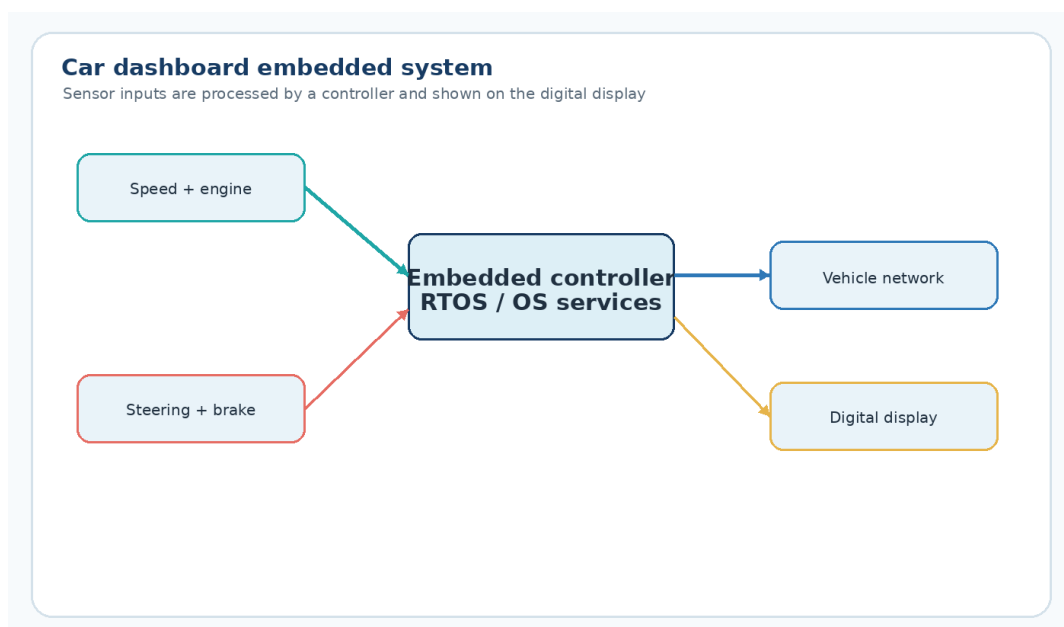


Figure 15: A car dashboard embedded system reading sensors and driving the digital display

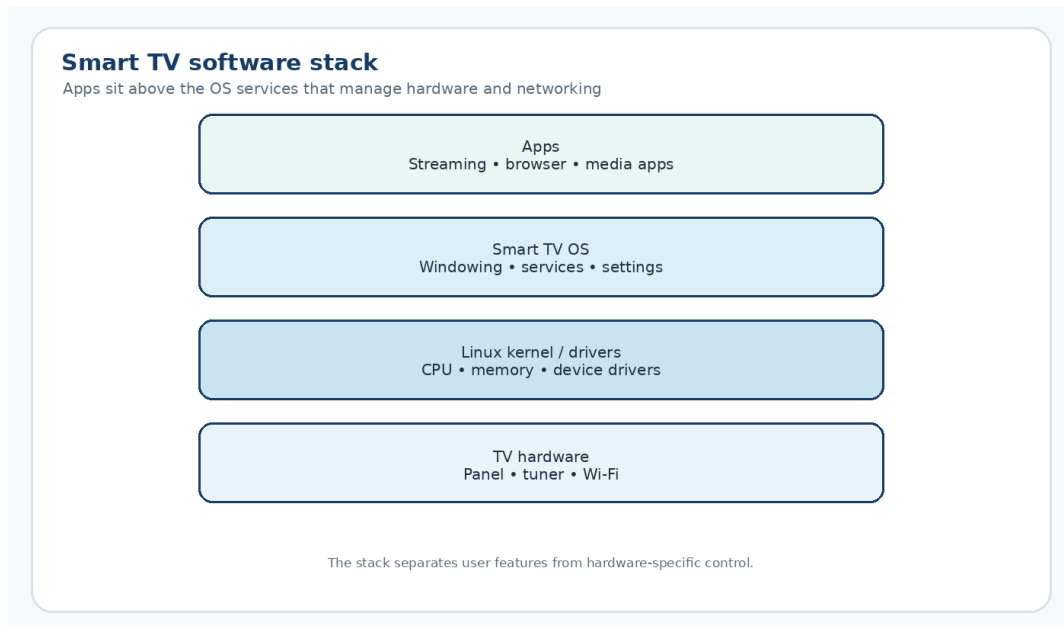


Figure 16: Software stack of a typical smart TV built on an embedded operating system

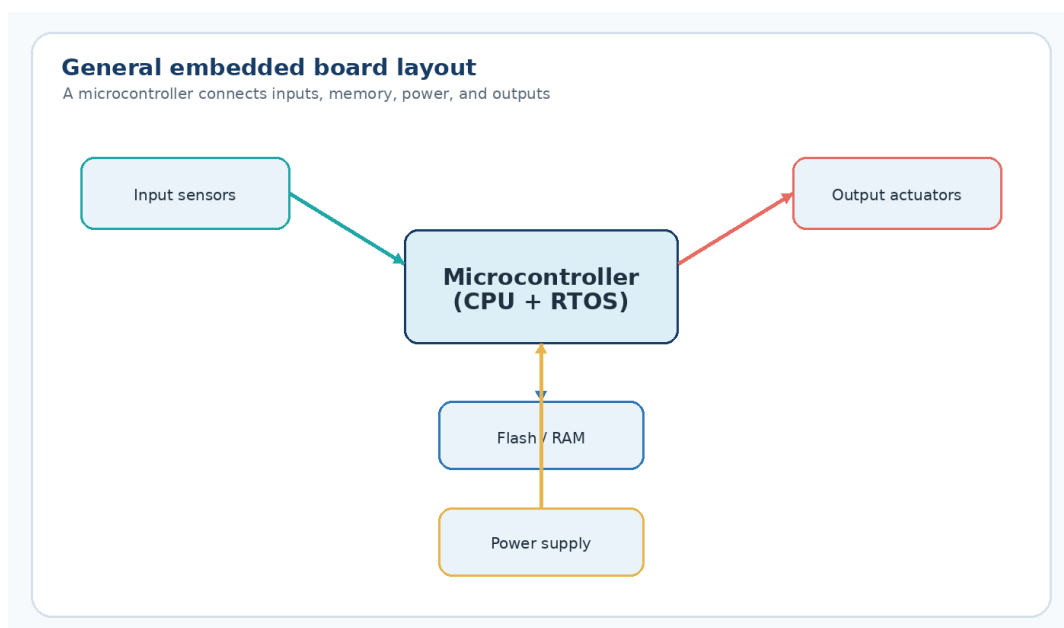


Figure 17: A general embedded board layout with a microcontroller, sensors, and actuators

3.14 Comparison Tables

The following tables give a compact comparison of the main operating system types and products discussed in the three case studies.

Point	RTOS	IoT OS	Mobile OS
Main Goal	Meet strict deadlines	Save power, stay small	Support many apps and users
Typical Memory	Kilobytes	Kilobytes	Gigabytes
Example	RTOS in a hospital monitor	FreeRTOS, Zephyr	Android, iOS

Table 3: RTOS vs IoT OS vs Mobile OS

Point	Android	iOS
Base	Built on the Linux kernel	Built on a Unix-based Darwin kernel
Openness	Open source, many device makers	Closed source, only Apple devices
App Store	Google Play	Apple App Store
Customization	Highly customizable	More restricted, fewer settings exposed

Table 4: Android vs iOS

Point	FreeRTOS	Zephyr
Footprint	Extremely small	Small to medium
Networking	Added through extra libraries	Built-in networking stack
Best Fit	Very simple sensors and wearables	Devices needing Bluetooth or Wi-Fi out of the box
Backed By	Amazon Web Services	The Linux Foundation

Table 5: FreeRTOS vs Zephyr

3.15 Advantages

- Mobile operating systems support many applications and a smooth user experience.
- Embedded operating systems can fit small, low-cost hardware.
- Modern systems combine real-time and power-saving techniques with broader OS features.
- Large developer communities and mature tools support application and device development.

3.16 Challenges

- Battery life and performance must be balanced carefully.
- Android devices may use different OS versions and hardware configurations.
- Older or low-cost embedded devices may receive fewer security updates.

3.17 Summary

Mobile and embedded operating systems are designed for different workloads. Android and iOS focus on apps, user experience, battery life, and security. Embedded systems such as FreeRTOS or embedded Linux focus more on predictable work on limited hardware. The examples of smartphones, smartwatches, dashboards, and smart TVs show that processes, threads, scheduling, memory, and security are common themes across both areas.

References

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
- [2] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. Boston, MA, USA: Pearson, 2018.
- [3] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Boston, MA, USA: Pearson, 2015.
- [4] Amazon Web Services, "FreeRTOS Documentation," FreeRTOS. [Online]. Available: <https://www.freertos.org>. [Accessed: Jul. 6, 2026].
- [5] Zephyr Project, "Zephyr Project Documentation." [Online]. Available: <https://www.zephyrproject.org>. [Accessed: Jul. 6, 2026].
- [6] Google, "Android Developers Documentation." [Online]. Available: <https://developer.android.com>. [Accessed: Jul. 6, 2026].
- [7] Apple Inc., "Apple Developer Documentation." [Online]. Available: <https://developer.apple.com>. [Accessed: Jul. 6, 2026].
- [8] The Linux Kernel Organization, "The Linux Kernel Archives." [Online]. Available: <https://www.kernel.org>. [Accessed: Jul. 6, 2026].
- [9] RIOT OS Community, "RIOT OS Documentation." [Online]. Available: <https://riot-os.org>. [Accessed: Jul. 6, 2026].
- [10] Contiki-NG Community, "Contiki-NG Documentation." [Online]. Available: <https://contiki-ng.org>. [Accessed: Jul. 6, 2026].
- [11] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," Journal of the ACM, vol. 20, no. 1, pp. 46–61, Jan. 1973.
- [12] G. C. Buttazzo, Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications, 3rd ed. New York, NY, USA: Springer, 2011.